

Étape 2 – Comprendre les données et modéliser la base

*Projet SportDataPulse – Analyse des performances des joueurs et des équipes
Ligue 1 – Saison 2025*

Ce livrable a pour objectif d’analyser la structure des fichiers de données fournis, d’évaluer leur qualité et de concevoir un modèle logique de données adapté aux besoins métier identifiés lors de l’étape 1.

Il présente :

- une compréhension des sources de données et de leur organisation,
 - une analyse de la qualité et des éventuelles anomalies,
- une proposition de modélisation relationnelle cohérente,
 - un dictionnaire de données détaillé,
- ainsi que les règles de transformation et de nettoyage appliquées.

Ce document constitue une étape clé avant la création de la base de données et la réalisation des analyses SQL.

1. Périmètre de l'étape 2

Cette étape a pour objectif de comprendre la structure réelle des fichiers de données fournis, d'identifier les entités exploitables et de proposer un modèle relationnel cohérent en vue de la création de la base de données.

L'analyse des fichiers met en évidence un point structurant du projet : contrairement au brief initial, les données ne sont pas disponibles au niveau du match ou de l'événement (tir, passe, action), mais au niveau agrégé du joueur sur une saison complète.

Le grain des données est donc défini comme suit :

- 1 ligne = 1 joueur pour la saison 2025

Ce choix de grain implique une modélisation spécifique :

- une table centrale « joueur »,
- des tables de référence (équipe, position),
- des tables statistiques spécialisées (performance, temps de jeu, attendu, progression),
- et une table complémentaire pour les salaires.

Le modèle de données est ainsi conçu pour permettre l'analyse statistique globale des performances, et non l'analyse fine des événements de match.

2. Inventaire des feuilles analysées

Feuille	Nb lignes utiles	Nb colonnes utiles	Principaux champs	Identifiant repéré
joueur	540	6	id_joueur, code_position, id_equipe, nom_joueur, nation, annee_naissance	id_joueur
temps_de_jeu	540	5	id_joueur, matches_joues, matches_titulaires, minutes_jouees, matches_90	id_joueur
performance	540	8	id_joueur, buts, passes_decisives, buts_hors_penalties, penalties_marques, penalties_tentes, cartons_jaunes, cartons_rouges	id_joueur
attendu	540	4	id_joueur, xg, npxg, xag	id_joueur
progression	540	4	id_joueur, progression_conduite, progression_passe, progression_course	id_joueur
position	16	2	code_position, libelle_position	code_position
equipe	18	2	id_equipe, nom_equipe	id_equipe
salaire_source	545	5	id_source, nom_joueur, age, salaire_annuel, note_source	id_source

3. Constats de qualité des données

L’analyse des fichiers de données après nettoyage met en évidence les éléments suivants :

- Les tables principales (joueur, temps_de_jeu, performance, attendu, progression) contiennent chacune 540 lignes, correspondant à un joueur unique pour la saison 2025. Les données sont cohérentes entre elles et permettent des jointures fiables.
- Les tables de référence (equipe et position) sont de petite taille (18 équipes, 16 positions) et servent de dimensions pour enrichir les analyses.
- La table salaire_source contient 545 lignes. Elle présente un écart de volumétrie avec la table joueur et ne partage pas directement le même identifiant. Une jointure nécessite donc une étape de réconciliation (par nom ou autre clé intermédiaire), ce qui introduit un risque d’erreur.
- L’identifiant id_joueur est cohérent et présent dans toutes les tables principales. Il constitue la clé primaire et la clé de jointure principale du modèle relationnel.
- Le nom du joueur ne peut pas être utilisé comme identifiant unique, en raison de la présence potentielle d’homonymes. Il est utilisé uniquement à des fins d’affichage.
- Les données ont été standardisées (noms de colonnes, formats numériques), ce qui permet une exploitation directe en SQL sans transformation supplémentaire.
- Certaines colonnes présentes dans les fichiers sources ont été supprimées car redondantes ou calculables (ex : buts + passes décisives, xG + xAG). Ces indicateurs sont désormais calculés dynamiquement dans la base de données.
- Le jeu de données ne contient pas d’informations temporelles fines (match par match), ce qui limite l’analyse à un niveau agrégé par joueur sur la saison.

4. Modèle logique recommandé

5. Relations retenues

Table source	Table cible	Cardinalité	Justification
position	joueur	1-N	Un code de position peut être associé à plusieurs joueurs.
equipe	joueur	1-N	Une équipe regroupe plusieurs joueurs.
joueur	temps_de_jeu	1-1	Une seule ligne de temps de jeu par joueur pour la saison 2025.
joueur	performance	1-1	Une seule ligne de performance agrégée par joueur pour la saison 2025.
joueur	attendu	1-1	Une seule ligne de statistiques attendues par joueur pour la saison 2025.
joueur	progression	1-1	Une seule ligne d’indicateurs de progression par joueur pour la saison 2025.
joueur	salaire_source	0-1	Lien optionnel nécessitant une réconciliation préalable des identifiants et une validation métier.

Remarque :

Les relations 1-1 entre la table joueur et les tables statistiques sont justifiées par le grain des données (1 joueur = 1 ligne par saison). Dans un modèle intégrant plusieurs saisons ou des données match par match, ces relations évolueraient vers des cardinalités 1-N.

6. Dictionnaire de données cible

Le dictionnaire ci-dessous décrit la structure cible de la base, prête pour la création SQL.

POSITION

Champ	Type SQL conseillé	Contraintes	Description
code_position	VARCHAR	PK	Code de position principal (GB, DF, MT, AT).
libelle_position	VARCHAR	NOT NULL	Libellé de la position.

EQUIPE

Champ	Type SQL conseillé	Contraintes	Description
id_equipe	INTEGER	PK	Identifiant unique du club.
nom_equipe	VARCHAR	NOT NULL, UNIQUE	Nom du club.

JOUEUR

Champ	Type SQL conseillé	Contraintes	Description
id_joueur	INTEGER	PK	Identifiant joueur issu des données sources.
nom_joueur	VARCHAR	NOT NULL	Nom complet du joueur.
nation	VARCHAR	NULL	Nationalité du joueur.
annee_naissance	INTEGER	NULL	Année de naissance.
code_position	VARCHAR	FK -> POSITION(code_position)	Poste principal.
id_equipe	INTEGER	FK -> EQUIPE(id_equipe)	Club d'appartenance.

TEMPS_JEU

Champ	Type SQL conseillé	Contraintes	Description
id_joueur	INTEGER	PK, FK -> JOUEUR(id_joueur)	Identifiant du joueur.
matches_joues	INTEGER	CHECK >= 0	Nombre de matchs joués.
matches_titulaires	INTEGER	CHECK >= 0	Nombre de matchs débutés.
minutes_jouees	INTEGER	CHECK >= 0	Temps total en minutes.
matches_90	NUMERIC	CHECK >= 0	Équivalent nombre de matchs complets.

PERFORMANCE

Champ	Type SQL conseillé	Contraintes	Description
id_joueur	INTEGER	PK, FK -> JOUEUR(id_joueur)	Identifiant du joueur.
buts	INTEGER	CHECK >= 0	Nombre total de buts.
passes_decisives	INTEGER	CHECK >= 0	Nombre de passes décisives.
buts_hors_penalties	INTEGER	CHECK >= 0	Buts hors penalties.
penalties_marques	INTEGER	CHECK >= 0	Penalties marqués.
penalties_tentes	INTEGER	CHECK >= 0	Penalties tentés.
cartons_jaunes	INTEGER	CHECK >= 0	Cartons jaunes.
cartons_rouges	INTEGER	CHECK >= 0	Cartons rouges.
Champ	Type SQL conseillé	Contraintes	Description

Remarque :

L'indicateur buts_plus_pd (buts + passes décisives) est calculé dynamiquement en base de données afin d'éviter toute redondance.

ATTENDU

Champ	Type SQL conseillé	Contraintes	Description
id_joueur	INTEGER	PK, FK -> JOUEUR(id_joueur)	Identifiant du joueur.
xg	NUMERIC	CHECK >= 0	Expected Goals.
npvg	NUMERIC	CHECK >= 0	Expected Goals hors penalties.
xag	NUMERIC	CHECK >= 0	Expected Assists.

Remarque :

L'indicateur npvg_xag (npvg + xag) est calculé dynamiquement en base de données.

PROGRESSION

Champ	Type SQL conseillé	Contraintes	Description
id_joueur	INTEGER	PK, FK -> JOUEUR(id_joueur)	Identifiant du joueur.
progression_conduite	INTEGER	CHECK >= 0	Courses progressives.
progression_passe	INTEGER	CHECK >= 0	Passes progressives.
progression_course	INTEGER	CHECK >= 0	Passes reçues progressives.

SALAIRE_SOURCE

Champ	Type SQL conseillé	Contraintes	Description
id_source	INTEGER	PK	Identifiant de la source salaire.

nom_joueur	VARCHAR	NOT NULL	Nom du joueur dans la source.
age	INTEGER	NULL	Âge du joueur.
salaire_annuel	NUMERIC	CHECK >= 0	Montant du salaire annuel.
note_source	TEXT	NULL	Commentaire sur la qualité de la donnée.
id_joueur	INTEGER	NULL, FK -> JOUEUR(id_joueur)	Renseignement après réconciliation.
Champ	Type SQL conseillé	Contraintes	Description

7. Règles de modélisation et de nettoyage

1. Les données ont été préalablement nettoyées afin de supprimer les lignes vides et garantir un jeu de données exploitable. Les tables importées ne contiennent que des lignes utiles.
2. Les champs ont été renommés selon une convention homogène en base de données (minuscules, sans espaces ni accents) afin de garantir la lisibilité et la cohérence du modèle SQL.
3. Les colonnes redondantes présentes dans les fichiers sources (ex : 'ID équipe' et 'equipe' dans la table performance) ne sont pas conservées, car elles peuvent être déduites via les relations entre les tables (JOUEUR → EQUIPE).
4. Les indicateurs calculables (ex : buts + passes décisives, npxG + xAG) ne sont pas stockés en base afin d'éviter la redondance. Ils sont calculés dynamiquement lors des requêtes.
5. La table salaire_source est traitée comme une source externe nécessitant une étape de réconciliation. Le rapprochement avec la table joueur est effectué à l'aide de critères tels que le nom, l'âge et des contrôles manuels. En cas de correspondance incertaine, la donnée n'est pas intégrée afin de préserver la qualité des analyses.
6. Des contraintes de type CHECK sont appliquées sur les données numériques afin d'empêcher les valeurs incohérentes (ex : valeurs négatives).
7. Des tests de cohérence sont réalisés après chargement :
 - vérification du nombre de lignes par table,
 - validation des jointures entre les tables,
 - détection des valeurs nulles critiques,
 - contrôle des doublons éventuels.

8. Limites du modèle

Le modèle proposé repose sur des données agrégées à l'échelle de la saison 2025, avec un grain défini comme suit : 1 joueur = 1 ligne par table statistique.

Ce choix de modélisation permet des analyses globales des performances, mais présente plusieurs limites :

- Absence de données événementielles : le modèle ne permet pas d'analyser les actions de match (tirs, passes, dribbles, séquences de jeu).
- Absence de dimension temporelle : il est impossible d'étudier l'évolution des performances au cours de la saison (forme, progression, blessures).
- Analyse limitée à des indicateurs agrégés : les résultats ne permettent pas d'identifier précisément le contexte des performances (adversaire, situation de jeu, compétition).

Une évolution future du modèle nécessiterait l'intégration de données au niveau match ou événement, avec des tables supplémentaires (match, action, tir, passe), permettant une analyse plus fine et plus dynamique des performances.

9. Conclusion de l'étape 2

L'analyse des données et la modélisation réalisée permettent de concevoir une base relationnelle robuste, adaptée à l'étude des performances des joueurs de Ligue 1 sur la saison 2025.

Le modèle repose sur une table centrale JOUEUR, complétée par des tables statistiques spécialisées (temps_de_jeu, performance, attendu, progression) et des dimensions de référence (equipe, position). Cette structure garantit la cohérence des données, limite les redondances et facilite les analyses SQL.

Le principal point de vigilance concerne la table salaire_source, dont les identifiants ne sont pas directement compatibles avec ceux du fichier principal. Son intégration nécessite donc une étape de réconciliation préalable afin d'assurer la fiabilité des analyses de recrutement.

Ce modèle constitue une base solide pour la phase suivante du projet, dédiée à la création de la base de données, au chargement automatisé des données et à la réalisation des analyses SQL répondant aux besoins métier.